Hybrid Vision Model: Integrating Haar and YOLO for Robust Detection

1. Introduction

In the field of computer vision, object and face detection play a crucial role in various applications such as surveillance, biometric authentication, and automated attendance systems. Traditional models often focus on either speed or accuracy, leading to trade-offs in performance. To address this, we propose a **Hybrid Vision Model: Integrating Haar and YOLO for Robust Detection**, which leverages the strengths of both algorithms. The Haar cascade classifier ensures rapid face detection, while the YOLO (You Only Look Once) algorithm provides high-accuracy object detection. By combining these two techniques, our model achieves **real-time, multi-object detection with improved efficiency and precision**, making it a powerful solution for diverse computer vision tasks.

2. Objectives

- Develop a system that accurately detects faces in images.
- Provide a user-friendly web interface for image upload and detection.
- Implement object detection using YOLO, with ongoing improvements.
- Ensure the system is accessible and easy to use via a web-based application.

3. Technologies Used

- **Python**: Core programming language for implementing face and object detection.
- **OpenCV**: Used for image processing and face detection.
- **Haar Cascades**: Pre-trained model for detecting faces efficiently.
- **YOLO (Tiny YOLO v4)**: Used for object detection, with current improvements in accuracy.
- **Flask**: Lightweight web framework to create the user interface.
- **HTML, CSS, JavaScript**: For designing and enhancing the web interface.

What makes the proposed project unique?

1. **Combination of Techniques**: This model combines multiple algorithms (e.g., Haar cascades for face detection and YOLO for object detection), it qualifies as a **hybrid model** since it leverages different methods for improved accuracy.

2. **Advantages Over Existing Models**:
    - **Speed vs. Accuracy**: This model balances fast detection (Haar cascades) with high accuracy (YOLO), it's an optimized hybrid approach.
    - **Multiple Object Categories**: our model can detect **both faces and other objects simultaneously**, that's a unique feature.
    - **Customization & Optimization**: We are working on fine-tuning YOLO, applied pre-processing techniques, and optimized performance for specific applications (e.g., attendance tracking or security), which will add uniqueness.

In the proposed project, we are using OpenCV, Haar cascades, and YOLO for different purposes:

1. **Face Detection – Haar Cascade (OpenCV)**
    - we are using **Haar cascade classifiers** from OpenCV to detect faces in uploaded images.
    - Haar cascades work well for face detection because they are lightweight and efficient.
2. **Object Detection – YOLO (You Only Look Once)**
    - YOLO (Tiny YOLO v4) is being used to detect objects in images.
    - However, **object detection accuracy is still being improved**, as some objects are not being detected properly.
    - YOLO is more advanced than Haar cascades for general object detection but requires better tuning and training.
3. **Web Interface – Flask + HTML, CSS, JavaScript**
    - **Flask** is used to create a web-based application where users can upload images and view the detected objects/faces.
    - **HTML, CSS, JavaScript** are used to design and improve the front-end interface.

4. Implementation Details

- The system is designed to take an **image as input** via the web interface.
- **Face detection** is performed using OpenCV's Haar cascade classifier.

- **Object detection** is implemented using YOLO, though detection accuracy is still being optimized.
- The processed image with detected faces is displayed on the web interface.
- The user can provide feedback regarding detection accuracy.
- The system runs on a **Flask server**, making it accessible through a web browser.

5. User Interface and Functionality

- A **web-based interface** allows users to upload an image for detection.
- Users can view the processed image directly on the webpage.
- A **feedback section** is available for users to report detection accuracy.
- Copyright details and project information are displayed on the webpage.

6. Challenges and Improvements

- **Face detection works accurately**, but **object detection needs improvement**.
- Some small objects are not detected properly; we are **working on fine-tuning the YOLO model**.
- Future improvements include **training a custom model** for better accuracy.
- Enhancing the UI to include real-time detection capabilities.

7. Applications and Future Scope

- This system can be **extended for face recognition-based attendance tracking**.
- Integration with **real-time video feeds** for continuous face and object detection.
- Implementation of **deep learning models** for improved accuracy.
- Expansion into **mobile applications** for wider accessibility.

Working Procedure of the proposed Project:

1. Image Upload via Web Interface

- The user accesses the web-based interface built using **HTML, CSS, and JavaScript**.
- The interface provides an **"Upload & Detect"** button that allows users to select an image file from their device.
- Once the image is selected, it is temporarily displayed on the interface for preview.

2. Backend Processing (Flask Server)

- The Flask server, running in the background, listens for image upload requests.
- When the user uploads an image, the file is sent to the Flask backend via an HTTP POST request.
- The Flask application receives the image and saves it temporarily for processing.

3. Face Detection using OpenCV and Haar Cascades

- The uploaded image is **read using OpenCV (cv2.imread)**.
- **Image preprocessing** is performed (grayscale conversion, resizing if necessary).
- The Haar cascade classifier (haarcascade_frontalface_default.xml) is loaded and applied to detect faces in the image.
- If faces are detected, bounding boxes are drawn around them.

4. Object Detection using YOLO (Under Improvement)

- The YOLO model (yolov4-tiny.weights, yolov4-tiny.cfg) is loaded using OpenCV's Deep Neural Network (DNN) module.
- The image is preprocessed and fed into the YOLO network.
- The model predicts object classes and their locations.
- Bounding boxes are drawn around detected objects, though accuracy is still being optimized.

5. Sending the Processed Image Back to Frontend

- The processed image (with detected faces and objects) is saved.
- Flask returns the processed image to the front-end via an HTTP response.
- JavaScript updates the webpage to display the processed image with detected elements.

6. User Feedback Mechanism

- The user can provide feedback using a **text box** on the interface.
- If the detection is inaccurate, users can submit their comments.
- This feedback can be used later for improving the detection accuracy.

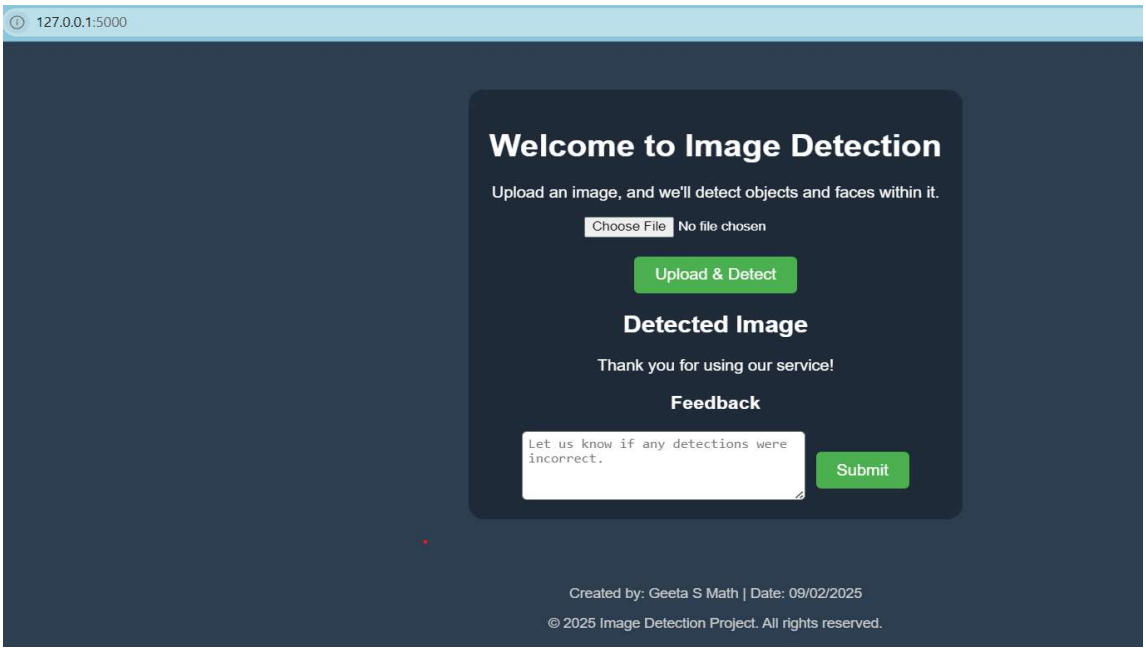
7. Deployment & Accessibility

- The Flask server runs locally (http://127.0.0.1:5000) for testing.
- To make the application accessible remotely, **port forwarding or cloud deployment** (e.g., AWS, Google Cloud) can be used.
- The project can be extended to support **real-time video detection** in future updates.

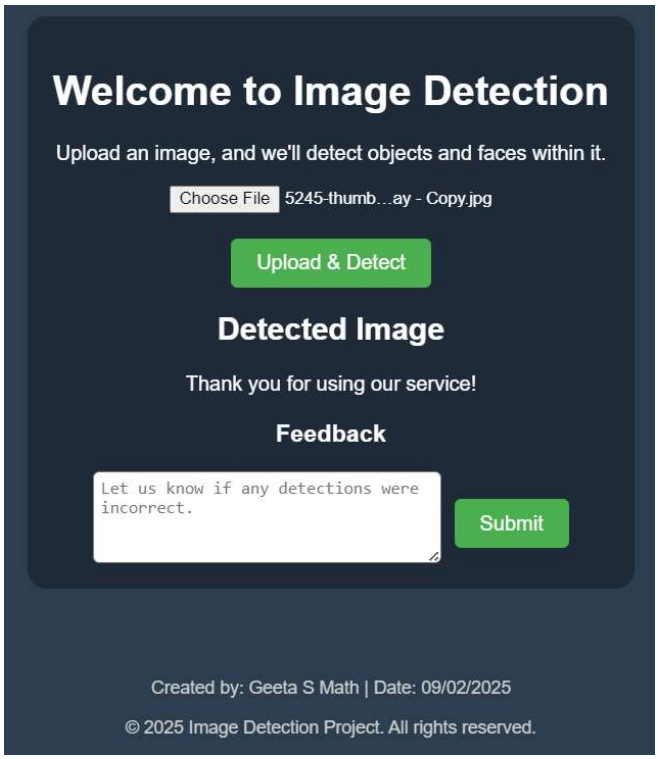
Live Working Model:

1. **Backend Processing (Flask Server)**

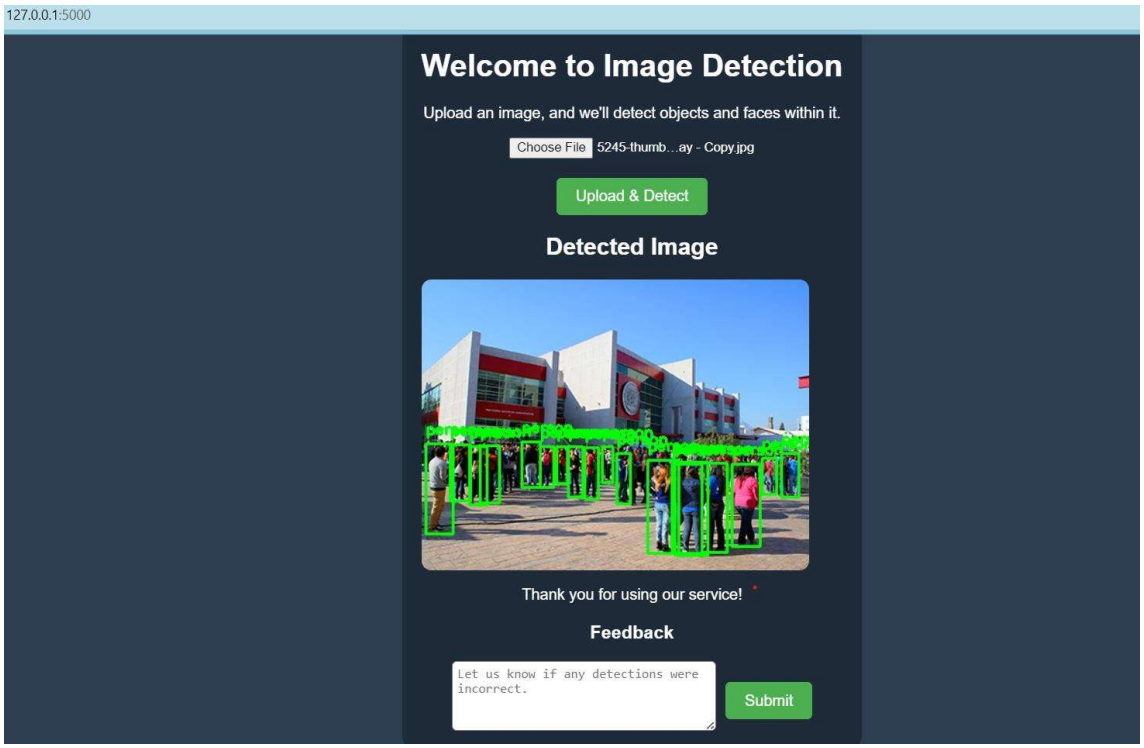
2. **Frontend Web Interface: http://127.0.0.1:5000/**



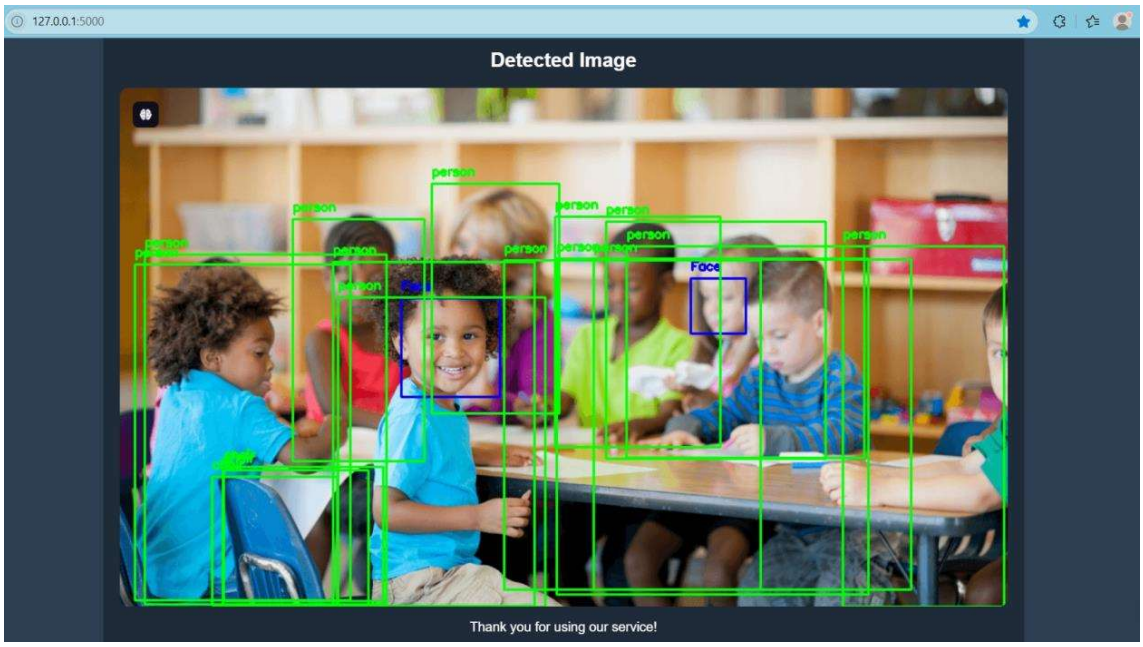
3. **Image Upload by choosing file via web interface:**
   Image is selected from the PC.



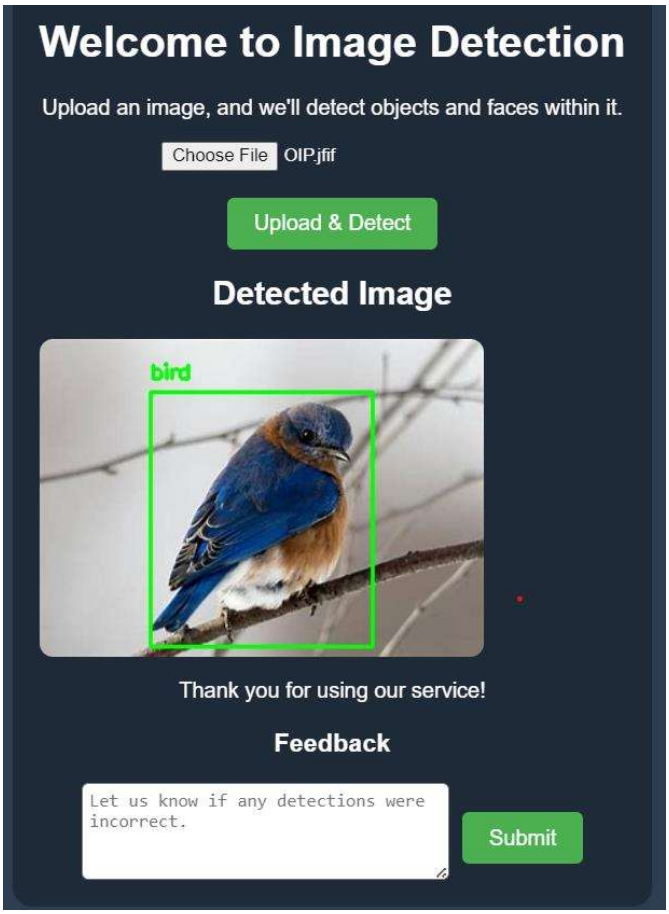
4. **Uploading selected image to detect face and objects.**
   Ex image 1:



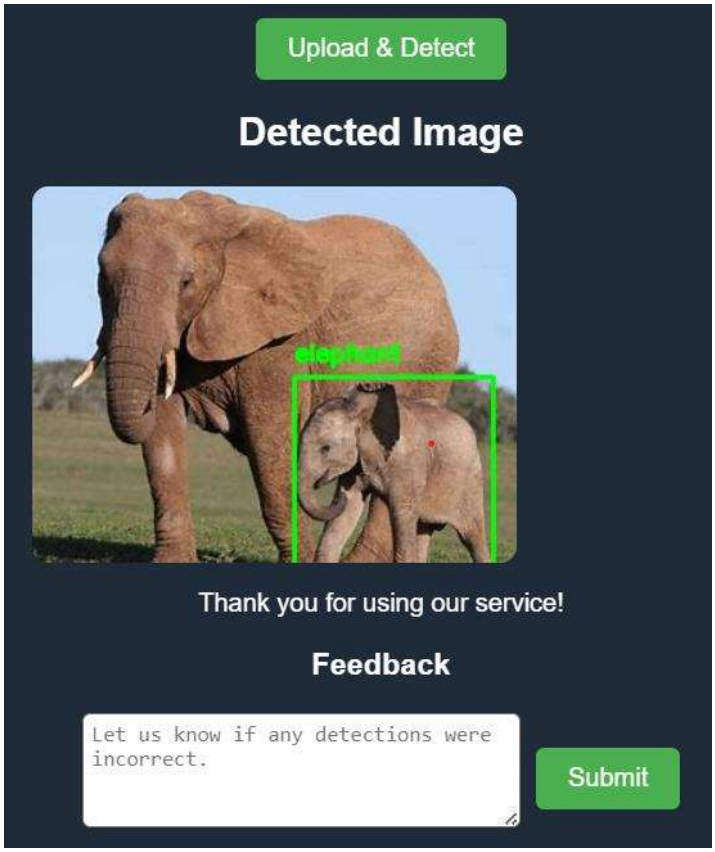
Ex Image 2:



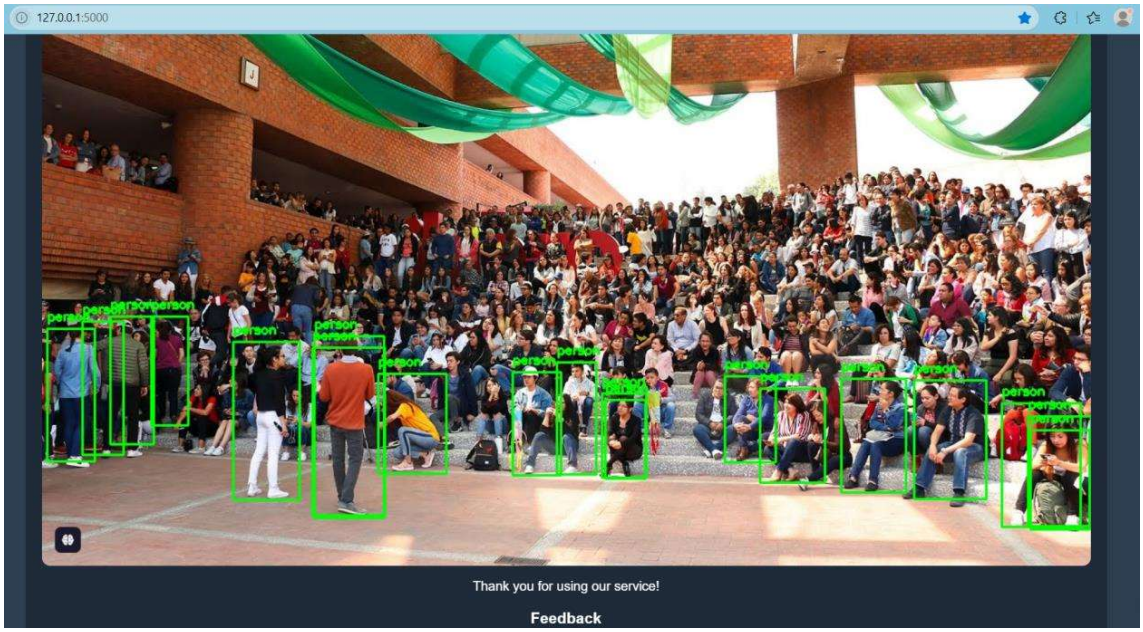
Ex Image 3:



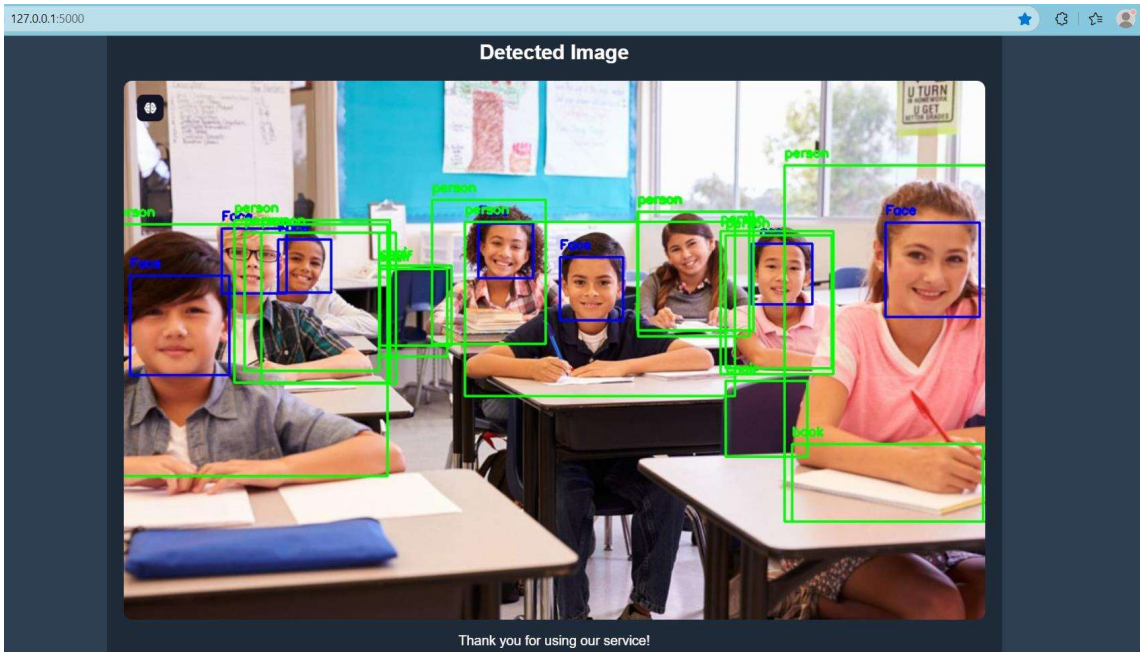
Ex Image 4:



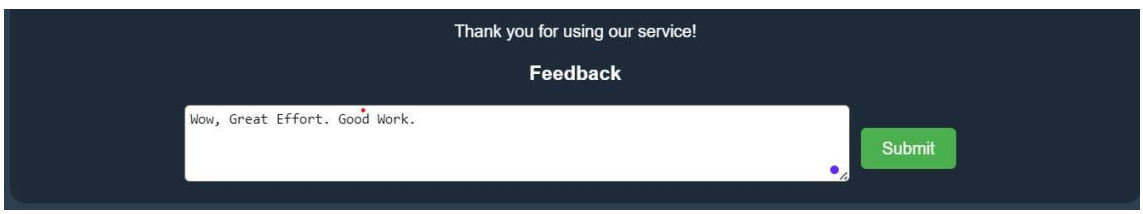
Ex Image 5:



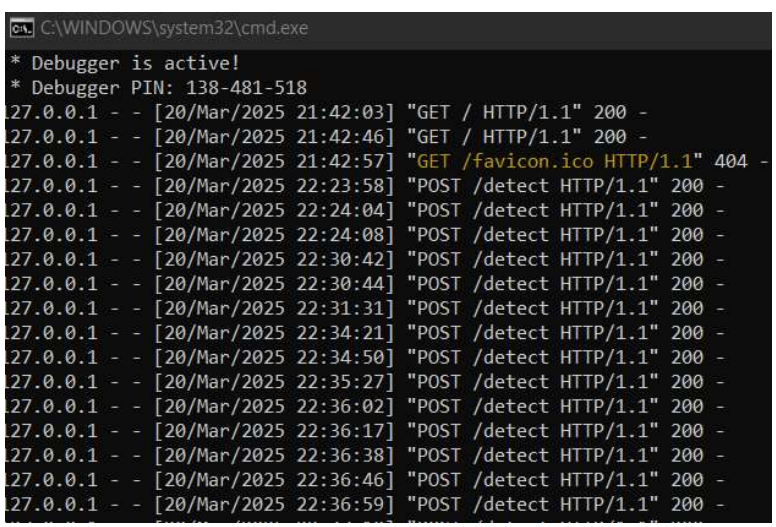
Ex Image 6:



5. **Feedback receiving from user:**



6. **Backend Logs:**



```
C:\WINDOWS\system32\cmd.exe
* Debugger is active!
* Debugger PIN: 138-481-518
127.0.0.1 - - [20/Mar/2025 21:42:03] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 21:42:46] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 21:42:57] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [20/Mar/2025 22:23:58] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:24:04] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:24:08] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:30:42] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:30:44] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:31:31] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:34:21] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:34:50] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:35:27] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:36:02] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:36:17] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:36:38] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:36:46] "POST /detect HTTP/1.1" 200 -
127.0.0.1 - - [20/Mar/2025 22:36:59] "POST /detect HTTP/1.1" 200 -
```

8. Conclusion :

In this work, we developed a **hybrid vision model** that combines Haar cascades for face detection and YOLO for object recognition, aiming to achieve a balance between **speed and accuracy**. While Haar cascades provide a lightweight solution for detecting faces in real-time, YOLO enhances object detection capabilities with its deep learning-based approach. This combination makes our model suitable for applications requiring **fast and efficient visual recognition**.

However, our analysis revealed certain **limitations**, such as Haar's sensitivity to lighting conditions and YOLO's occasional misdetections in smaller objects. These challenges highlight the need for further **optimization and fine-tuning** of detection parameters. Future work may involve replacing Haar with deep-learning-based face detection models or **training a custom YOLO model** to improve accuracy.

Overall, our hybrid approach provides an **effective and scalable** solution for real-time face and object detection, demonstrating its potential for use in **automated surveillance, attendance tracking, and smart vision applications**. 🚀

9. References

- You Only Look Once: Unified, Real-Time Object Detection
  Joseph Redmon∗, Santosh Divvala∗†, Ross Girshick, Ali Farhadi University of Washington∗, Allen Institute for AI†, Facebook AI Research http://pjreddie.com/yolo/

- Diwan, T., Anirudh, G. & Tembhurne, J.V. Object detection using YOLO: challenges, architectural successors, datasets and applications. *Multimed Tools Appl* **82**, 9243–9275 (2023). https://doi.org/10.1007/s11042-022-13644-y

- Peiyuan Jiang, Daji Ergu, Fangyao Liu, Ying Cai, Bo Ma, A Review of Yolo Algorithm Developments, Procedia Computer Science, Volume 199, 2022, Pages 1066-1073, ISSN 1877-0509, https://doi.org/10.1016/j.procs.2022.01.135

- Zou, X. (2019, August). A Review of Object Detection Techniques. In 2019 International Conference on Smart Grid and Electrical Automation (ICSGEA) (pp. 251-254). IEEE.

- Facial recognition using Haar cascade and LBP classifiers Anirudha B Shetty∗, Bhoomika, Deeksha, Jeevan Rebeiro, Ramyashree Dept. Computer Science and

Engineering, Shri Madhwa Vadiraja Institute of Technology and Management (Affiliated to VTU) Udupi, 574115 India

- S. P. Kambham et al., "Voice assistance to blind people by detection of surrounding objects, activities, and humans," 2024 International Conference on Computing and Intelligent Reality Technologies (ICCIRT), Coimbatore, India, 2024, pp. 16-21, doi: 10.1109/ICCIRT59484.2024.10921776.

- OpenCV and flask Documentation.